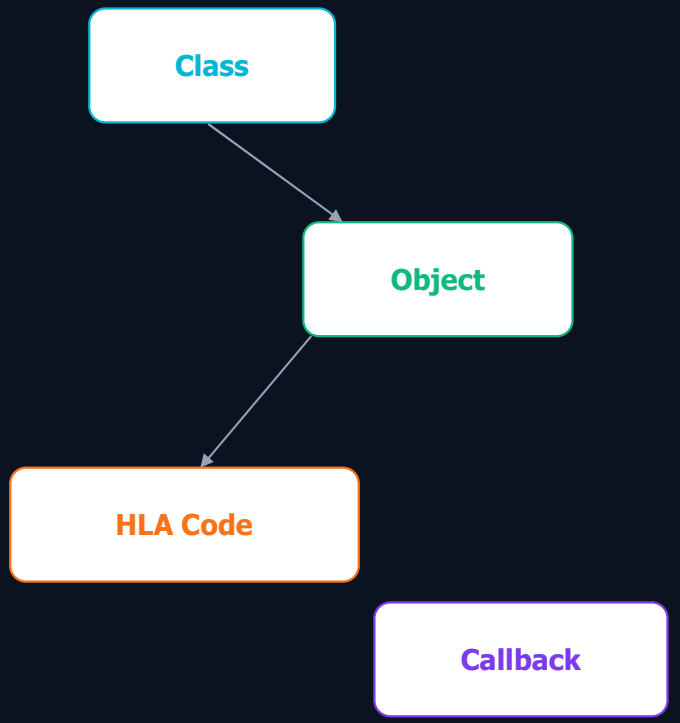
บทที่ 2

Java OOP Foundation

Class • Object • Encapsulation • Inheritance • Interface • Exception

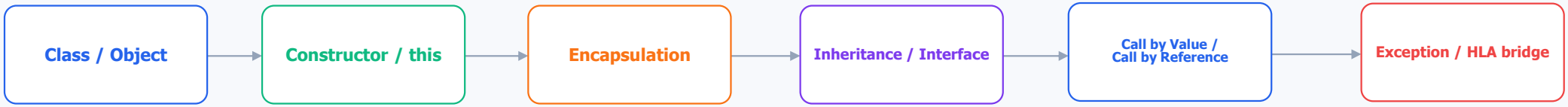
Call by Value • Call by Reference



บทที่ 2 ทำอะไรบ้าง

จาก "เขียน logic ได้" → "อ่านโครงสร้างโปรแกรมจริงได้"

OOP | HLA Prep



ผลลัพธ์หลังจบวันนี้

- สร้าง class/object เองได้
- อ่านโค้ด extends, @Override, interface ได้
- เข้าใจ private/getter/setter และ exception
- พร้อมเปิด HLA Federate code โดยไม่หลง syntax

```java
class PackageData {
    private String trackingNumber;

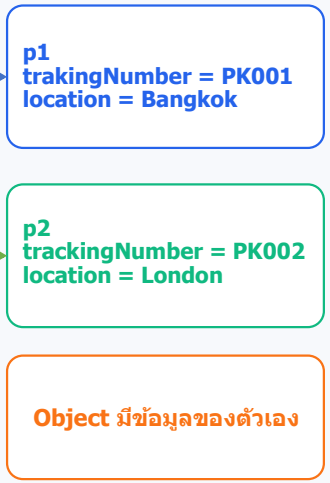

    public PackageData(String trackingNumber) {
        this.trackingNumber = trackingNumber;
    }
}
```

Class และ Object

Class คือแม่แบบ / Object คือของจริงที่ถูกสร้างขึ้น

```
class PackageData {
    String trackingNumber;
    String location;
}


PackageData p1 = new PackageData();
PackageData p2 = new PackageData();
```



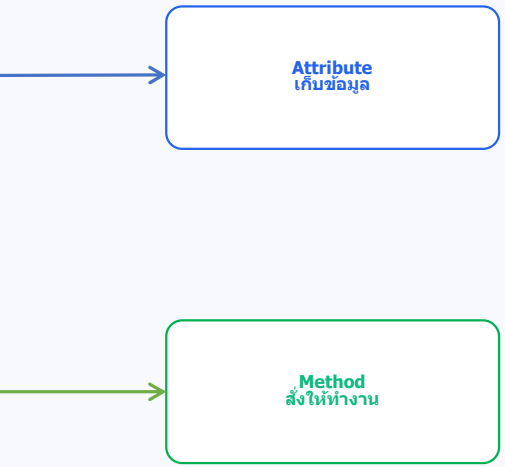
- HLA จะมี Object Instance หลายตัว
- Class ช่วยให้จัดข้อมูลเป็นรูปแบบเดียวกัน
- Object แต่ละตัวมี state ของตัวเอง

Attributes และ Methods

ข้อมูลอยู่ใน Attribute / พฤติกรรมอยู่ใน Method

```
class PackageData {
    String trackingNumber;
    String location;


    void updateLocation(String newLocation) {
        location = newLocation;
    }
}
```

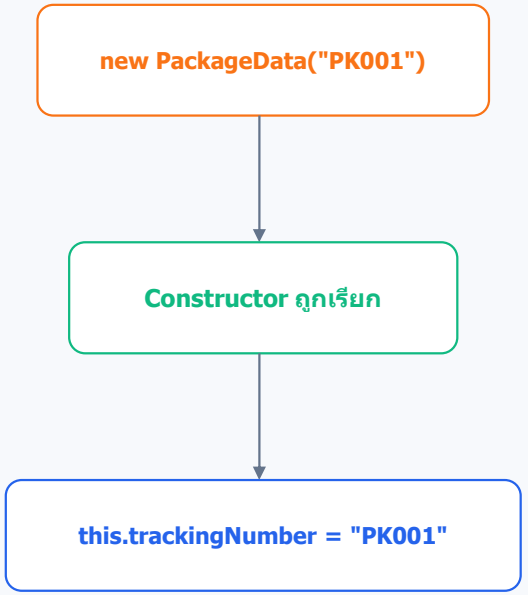


Constructor และ this

Constructor ทำงานตอน new Object / this หมายถึง object ตัวปัจจุบัน

```
class PackageData {
    private String trackingNumber;


    public SetPackageData(String trackingNumber) {
        this.trackingNumber = trackingNumber;
    }
}
```



- ใช้กำหนดค่าเริ่มต้นให้ Object
- ช่วยลด Object ที่ข้อมูลว่าง/ไม่สมบูรณ์
- ใน HLA มักมี constructor เตรียม ambassador / config

Encapsulation

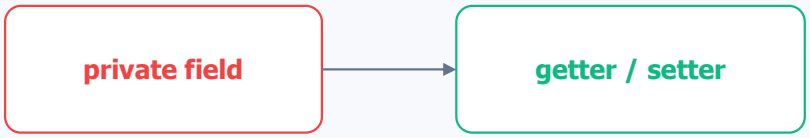
ปกป้องข้อมูลด้วย private แล้วเปิดทางเข้าผ่าน method ที่ควบคุมได้

```java
class PackageData {
	private String location;

	public String getLocation() {
		return location;
	}

	public void setLocation(String location) {
		this.location = location;
	}
}

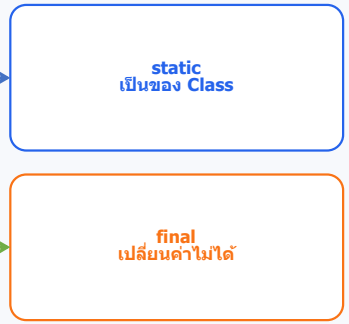
public class HelloWorld {
	public static void main(String[] args) {
		PackageData packageData = new PackageData();
		packageData.location = "Bangkok"; //Error
		packageData.setLocation("Bangkok");
		System.out.println(packageData.getLocation());
		//Bangkok
	}
}
```



- ห้ามแก้ข้อมูลจากข้างนอกมั่ว ๆ
- ควบคุม validation ได้ภายหลัง
- ทำให้ code ใหญ่ ๆ อ่านง่ายขึ้น

static และ final

```
class Config {
    static int packageCount = 0;
    static final String FEDERATION_NAME = "HLA_DEMO";
}
```



ตัวอย่างที่คล้าย HLA

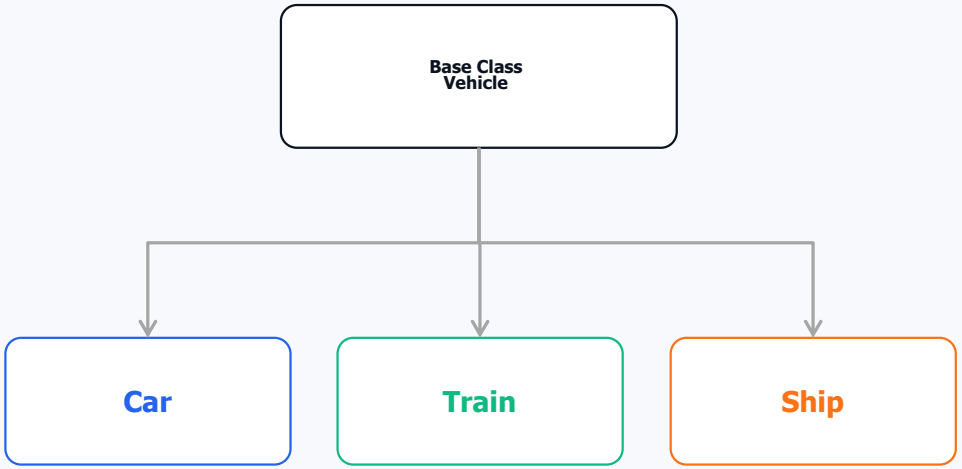
```
public class HelloWorld {
     public static final String READY_TO_RUN = "ReadyToRun";
     public static void main(String[] args) {
     // จุดเริ่มโปรแกรม

     }
}
```

- static main() คือจุดเริ่มโปรแกรม
- ค่าคงที่ใช้ static final
- ลด magic text ในโค้ด HLA

Inheritance: extends

Class ลูกสืบทอดความสามารถจาก Class แม่




```
class MyFederateAmbassador
    extends NullFederateAmbassador {
    ...
}
```

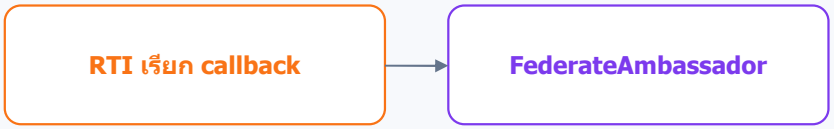
- extends = เอาความสามารถพื้นฐานมาใช้ต่อ
- ช่วยให้ Override เฉพาะสิ่งที่ต้องการ
- แนวนี้พบเยอะในตัวอย่าง HLA FederateAmbassador

Polymorphism และ @Override

Method ชื่อเดียวกัน แต่ class ลูกสามารถกำหนดพฤติกรรมใหม่ได้

```
class Animal {
    void speak() {
      System.out.println("...");
    }
}


class Dog extends Animal {
    @Override
    void speak() {
      System.out.println("Bark");
    }
}
```




```
@Override
public void receiveInteraction(...) {
    // code ที่เราต้องการให้ทำ
}
```

- @Override ช่วยบอกว่าเรากำลังเขียนทับ method เดิม
- ถ้าชื่อ method/parameter ผิด IDE จะเตือน
- สำคัญมากตอนเขียน HLA callback

Interface

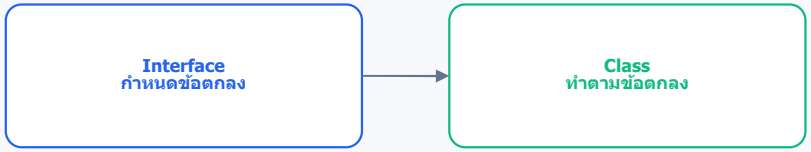
Interface คือ "ข้อกำหนด" ว่า class นี้ต้องมี method อะไรบ้าง

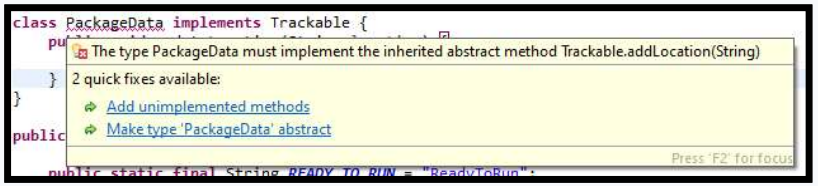
```java
interface Trackable {
    void updateLocation(String location);

    void addLocation(String location);
}


class PackageData implements Trackable {
    public void updateLocation(String location) {
        ...
    }

    public void addLocation(String location) {
        ...
    }
}
```





ถ้าเรียกใช้ Method ไม่ครบตามที่กำหนด จะมี Error แจ้งเตือน

Call by Value & Call by Reference in Java

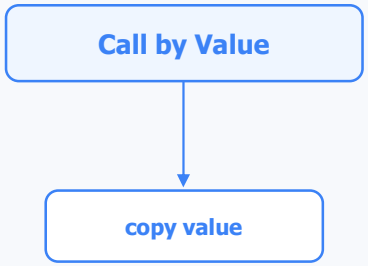
ภาพรวม: วิธีส่งข้อมูลเข้า Method และผลกระทบต่อค่าต้นทาง



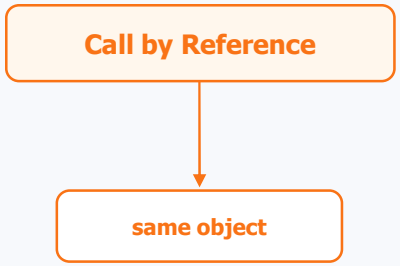
Method คือพื้นที่ทำงานย่อย
ที่รับข้อมูลผ่าน parameter

```
static void update(...){
    // parameter
}

update(argument);
```



เหมาะกับ primitive
เช่น int, double, boolean, char



ใช้กรณี object
ตัวแปรหลายตัวชี้ข้อมูลก้อนเดียวกัน

Wording

Parameter ตัวแปรที่ประกาศใน method

Argument ค่าที่ส่งเข้า method ตอนเรียกใช้

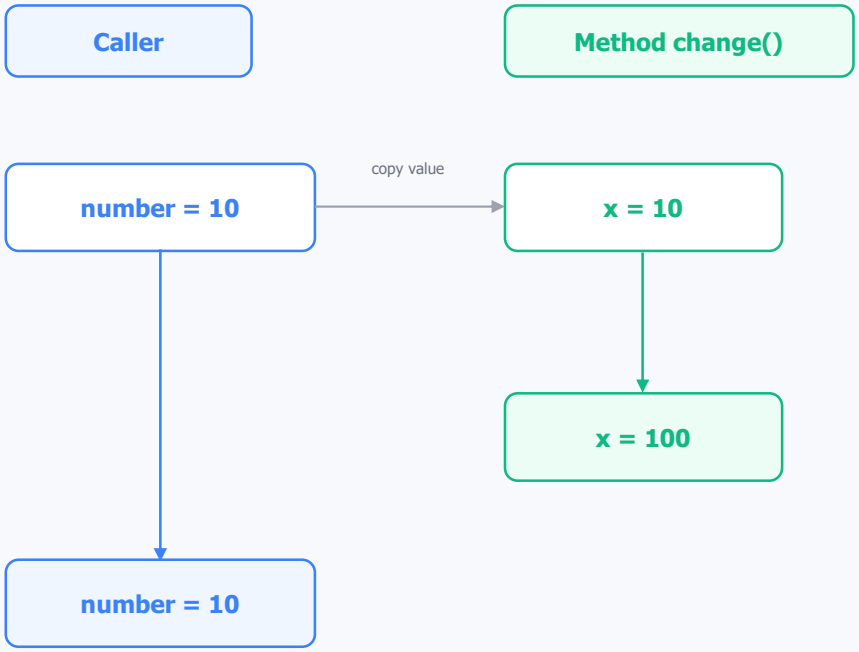
Call by Value

Primitive + diagram + code: Method ได้สำเนาของค่าไปทำงาน

```java
static void change(int x) {
    x = 100;
}

int number = 10;
change(number);

System.out.println(number); // 10
```



ค่าเดิมไม่เปลี่ยน

แนวคิดสำคัญ

- number เดิมอยู่ฝั่ง caller
- x คือ parameter ที่รับค่า copy
- เปลี่ยน x ไม่กระทบ number

Primitive ที่ควรรู้: int, double, boolean, char

Call by Reference

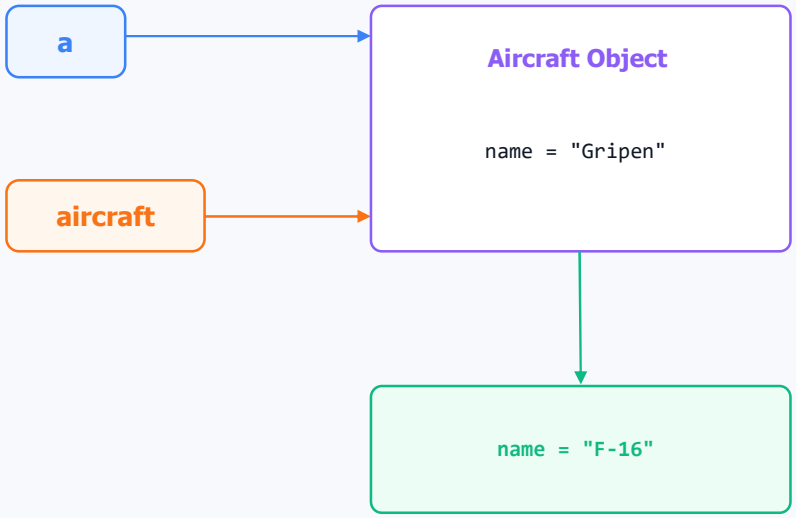
Object + diagram + code: ตัวแปรหลายตัวชี้ไปยัง Object เดียวกัน

```java
class Aircraft {
    String name;
}

public class HelloWorld {

    static void change(Aircraft aircraft) {
        System.out.println("Before: " + aircraft.name); //Gripen
        aircraft.name = "F-16";
    }

    public static void main(String[] args) {
        Aircraft a = new Aircraft();
        a.name = "Gripen";
        change(a);
        System.out.println("After: " + a.name); //F-16
    }
}
```



ทั้ง a และ aircraft ชี้ไปยัง Object ก้อนเดียวกัน

ผลลัพธ์ที่เห็น

เปลี่ยนผ่าน parameter แล้ว caller เห็นค่าที่เปลี่ยน เพราะชี้ Object เดียวกัน

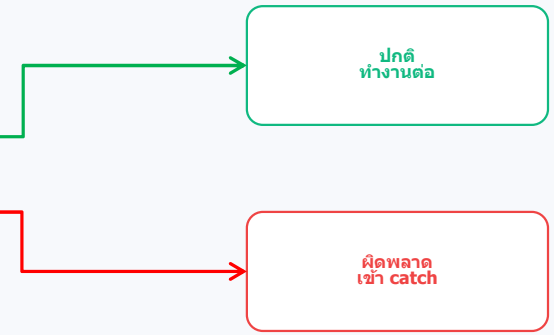
Tip: Strictly speaking, Java is always Pass by Value. สำหรับ Object ค่าที่ส่งคือสำเนาของ Reference ที่ชี้ไป Object เดียวกัน

Exception Handling

try-catch ช่วยให้โปรแกรมรับมือ error โดยไม่ Error จบหยุดการทำงานไป

```java
public static void main(String[] args) {
    try {
        int number = Integer.parseInt(input);
        System.out.println(number);
    } catch (NumberFormatException e) {
        System.out.println("Invalid number");
    }


}
```




```java
public void runFederate() throws Exception {
// HLA code หลายส่วนโยน exception ได้
}
```